

一. 选择

1. ARM Cortex-M3 不可以通过()唤醒CPU

- ☒ A I/O端口
- ☐ B RTC 闹钟
- ☐ C USB唤醒事件
- ☐ D PLL

第四章 最小系统: CPU 唤醒

2. 下面寄存器中, () 支持PWM互补输出

- ☒ A TIM1
- ☐ B TIM3
- ☐ C TIM5
- ☐ D TIM7

第六章 定时器 高级定时器TIM1、TIM8 支持 PWM 互补输出.

3. 以下哪个寄存器保存了当前激活中断的中断号 (A)。

- A. xPSR
- B. CONTROL
- C. CFGR
- D. BASEPRI

第三、六章均有,
特殊寄存器.

程序状态寄存器在其内部又被分为三个子状态寄存器:

- 应用程序 PSR (APSR)
- 中断号 PSR (IPSR)
- 执行 PSR (EPSR)

4. 10. 数值 a 和 b 的十六进制表示均为 0x1053CD68, 其中 a 是 32 位整数, b 是单精度浮点数, 那么 (A)。

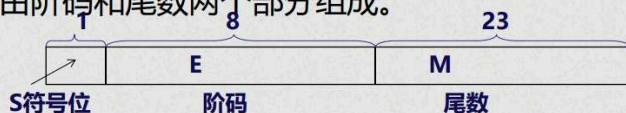
- A. a > b
- B. a < b
- C. a = b
- D. 无法判断

最后一个选择考察类似, 给出一个 8 位的 16 进制数. 0xDDDDDDDD. 比较它是补码、原码、单精度浮点数三个状态时, 对应的十进制数的大小.

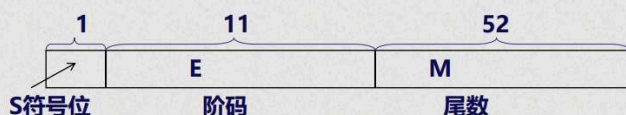
浮点数

IEEE754 标准浮点数由阶码和尾数两个部分组成。

单精度格式: 32 位

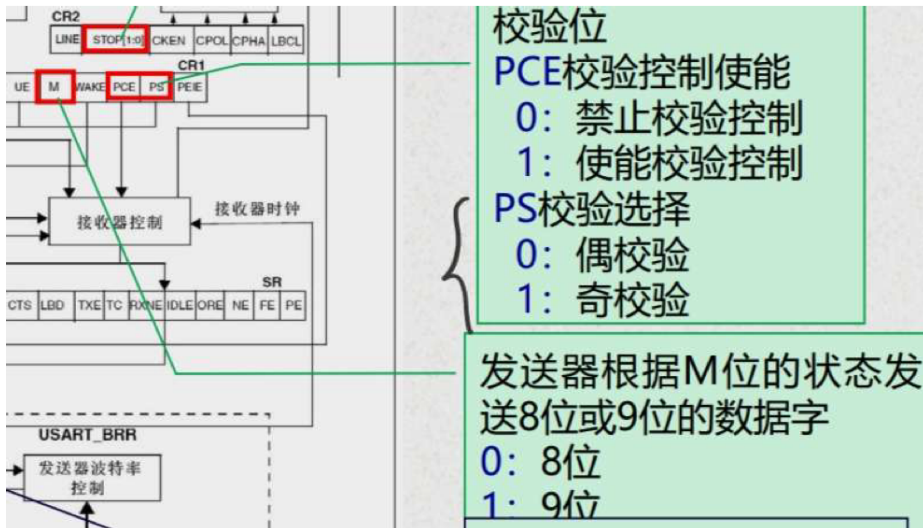


双精度格式: 64 位



补码 = 原码 (除符号位) 取反 + 1.

5. 第六章. USART串口, 奇偶校验, 考察写入"D", 读出来是多少.
涉及数据8/9位, MSB与LSB, USART寄存器①



6. 某处理器有30根地址线, 可寻址地址空间为 (C)。

A. 16M

B. 256M

C. 1G

D. 4G

7. CM3 微处理器的特点

使得可并行进行

- (1) 三级流水线设计; (取址, 译码, 执行)
- (2) 哈佛总线架构;
- (3) 32位寻址, 支持4G存储器空间;
- (4) 基于ARM AMBA技术的片上接口, 支持高吞吐量的流水线总线操作;
中断中执行更高优先级中断
- (5) 嵌套向量中断控制器 (NVIC), 支持11个系统异常, 最多240个中断请求, 以及8~256个中断优先级;
- (6) 支持操作系统的多种特性;

8. STM32 低功耗模式比较：待机模式下功耗最低

二、填空

1. -128 的补码 (4位16进制表示) = 0X0180

进制转化:

还有一个 (考察补码的转换)

(101.1)H

--> (257.0625) D

2. ASCII 码中的大、小写字母转换

例1: 用逻辑运算将ASCII码实现a~z和A~Z之间的转换

解答: 首先要清楚大写字母和小写字母的区别

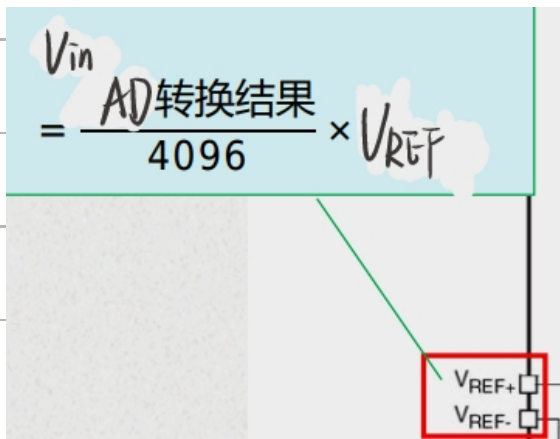
小写 'a' ~ 'z' 对应的ASCII为: 61H (01100001B) ~ 7AH (01101010B)

大写 'A' ~ 'Z' 对应的ASCII为: 41H (01000001B) ~ 5AH (01000010B)

小写→大写: 小写字母 &11011111B (DFH) →大写字母

大写→小写: 大写字母 | 00100000B (20H) →小写字母

9. 已知 ADC 采样的电压范围为 0V~+3.3V, ADC1->DR 的值为 0x418, 则实际电压值为 (0.84) V。



(第六章, AD转换)

4. 10. 在小端存储格式中，字数据 0x12345678 在内存中按字节地址由低到高依次存放为 (0x78 0x56 0x34 0x12)。

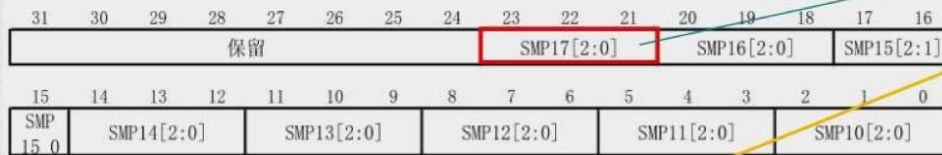
大端、小端存储格式(第三章)

5. A/D转换器的校准、数据对齐、采样时间设置及外部触发

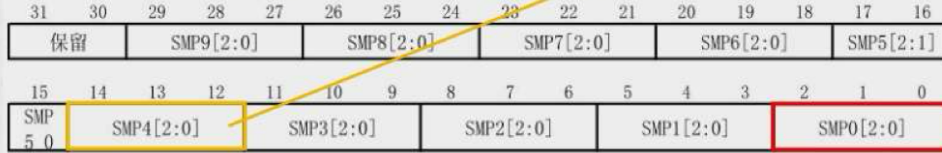
• 通道采样时间设置

ADC总转换时间计算: $T_{CONV} = \text{采样时间} + 12.5 \text{ 个周期}$

ADC 采样时间寄存器 1(ADC_SMPR1)



ADC 采样时间寄存器 2(ADC_SMPR2)



SMPx[2:0]: 选择通道x的采样时间

000: 1.5周期

001: 7.5周期

010: 13.5周期

011: 28.5周期

100: 41.5周期

101: 55.5周期

110: 71.5周期

111: 239.5周期

例: 当ADCCLK=14MHz, SMP4[2:0]=3, 对应通道的总转换时间TCONV?

$$T_{CONV} = 28.5 + 12.5 = 41 \text{ 周期} \approx 2.93 \mu s$$

转换时间的计算

6. AFIO 的配置

例: 设置PC13作为外部中断请求输入线。

13号对应EXTI13, 而EXTI13在EXTICR[3]上。让EXTI13[3:0]=0010, 0010表示C口被设置位外部中断请求线。

C语言实现如下:

```
AFIO->EXTICR[3] &= 0xFF0F; //清除EXTI13[3:0]
```

```
AFIO->EXTICR[3] |= 0x0020; //设置EXTI13[3:0]为0010
```


三.判断

- 1. IPR 寄存器 AIRCR寄存器 (第1章.中断)
- 2. Lite OS. 延时时用函数 Ldg-TaskDelay (). 不可用 delay ()
- 3. Lite OS 信号量的作用.

信号量 (Semaphore) 是一种实现任务间通信的机制,

4. Lite OS. 互斥锁有优先继承机制.

5. 有关优先级的判断

◆异常类型

优先级最高 {

编号	类型	优先级	简介
1	复位	-3 (最高)	复位
2	NMI	-2	不可屏蔽中断 (来自外部NMI输入脚)
3	硬 (hard) fault	-1	所有被除能的fault, 都将“上升” (escalation)成硬 fault。只要FAULTMASK没有置位, 硬fault服务例程就被强制执行。Fault被除能的原因包括被禁用, 或者FAULTMASK被置位

- 优先级的数值越小，则优先级越高。CM3 支持中断嵌套，使得高优先级异常会抢占(preempt)低优先级异常。
- 有3个系统异常：复位，NMI 以及硬fault，它们有固定的优先级，并且它们的优先级号是负数，从而高于所有其它异常。
- 所有其它异常的优先级则都是可编程的（但不能编程为负数）。

四. 大题

1. 单精度浮点数转换. 最终表示为16进制.

思考: 将十进制数9转换为IEEE754标准的单精度数是多少? ,如果用补码来表示是多少?

解答: (1) 转换成IEEE754标准的单精度数

$$9 = (-1)^0 \times 1001 = (-1)^0 \times 2^3 \times 1.001 = (-1)^0 \times 2^{130-127} \times 1.001 \quad 130 D = 1000\ 0010 B$$

二进制代码为:

0 10000010 001000000000000000000000 即: 41100000H

(Handwritten notes: 130, 尾数, 共23位, 20位)

(2) 如果表示成补码

$$9 = 1001 \quad \text{即} 000000009H$$

$$(3) 13/32 = (-1)^0 \times 1101 \times 2^{-5} = (-1)^0 \times 2^{-5} \times 2^3 \times 1.101 = (-1)^0 \times 2^{125-127} \times 1.101$$

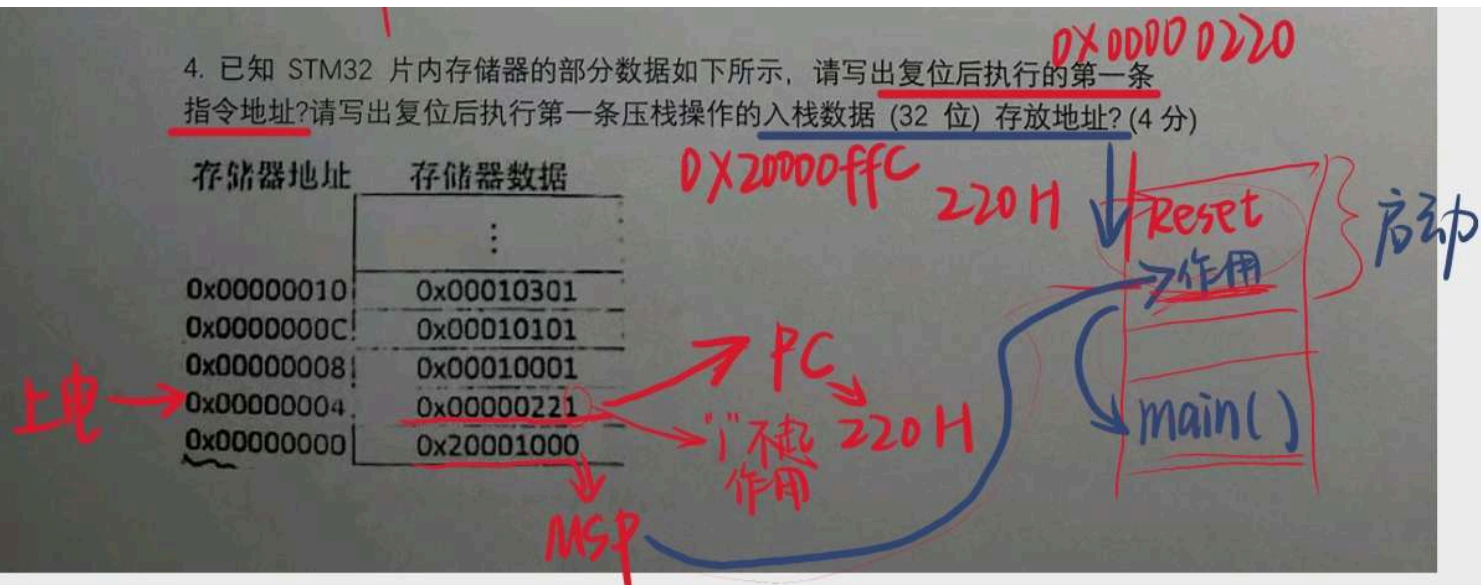
IEEE754编码为: 0 01111101 101000000000000000000000 即: 3ED00000H

$$\frac{1}{32} = 2^{-5}$$

2. (第四章) 复位.

4. 已知 STM32 片内存储器的部分数据如下所示, 请写出复位后执行的第一条指令地址? 请写出复位后执行第一条压栈操作的入栈数据 (32 位) 存放地址? (4 分)

存储器地址	存储器数据
	⋮
0x00000010	0x00010301
0x0000000C	0x00010101
0x00000008	0x00010001
0x00000004	0x00000221
0x00000000	0x20001000



上电后, 从从小到大第2个地址, 取复位程序的第1条指令. (注: 为了让单片机知道是Thumb模式运行, 最低位写1, 但读0), 第1个地址存放MSP的地址, STM32

是向下生成的满堆栈，第1条压栈操作的数据存放于(MSP地址-4)的地址。

3. (第三章) 位带与位带别名区。

位带地址映射与位带操作实例

◆ 位带区与位带别名区的映射关系

一共 $8 \times 4 = 32$ bit(位)

如位带区某个位 (bit)，所在的字节地址为 **A**，在字节中的位序号为 **n** ($0 \leq n \leq 7$)，则该位在别名区的地址为：

片上SRAM位带区：

$$\begin{aligned} \text{别名区地址} &= 0x22000000 + ((A - 0x20000000) \times 8 + n) \times 4 \\ &= 0x22000000 + (A - 0x20000000) \times 32 + n \times 4 \end{aligned}$$

片上外设位带区：

$$\begin{aligned} \text{别名区地址} &= 0x42000000 + ((A - 0x40000000) \times 8 + n) \times 4 \\ &= 0x42000000 + (A - 0x40000000) \times 32 + n \times 4 \end{aligned}$$

4. (第六章) 定时器，定时器时钟树，定时器寄存器

4. 设 SYSCLK=56MHz，AHB 时钟=56MHz，APB1 时钟=14MHz，则通用定时器 TIM3 的时钟 CK_INT 的频率是多少？TIM3 采用向上计数和时钟 CK_INT 实现 300ms 定时溢出，则寄存器 TIM3_PSC、TIM3_ARR 的值应分别设置为多少？

因为 AHB=56MHz，APB1 时钟=14MHz，预分频=56/14=4，所以定时器时钟位 $14 \times 2 = 28\text{MHz}$ 。

以下答案不唯一，参考答案：

TIM3_PSC=2799；

TIM3_ARR=3000。

或者

TIM3_PSC=27999；

TIM3_ARR=300。

$$\frac{28000000}{2799 + 1} = 10\text{kHz}$$

$$\text{APR} \times \frac{1}{10000} = 0.3, \text{APR} = 3000$$

5. BSRR寄存器的配置方法

置位复位寄存器BSRR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
W	W	W	W	W	W	W	W	W	W	W	W	W	W	W	W
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
W	W	W	W	W	W	W	W	W	W	W	W	W	W	W	W

例：把GPIOC的0位和2位置1， 14和15位复位

① 同时复位和置位：

```
GPIOC->BSRR = 0xC0000005;
```

② 复位置位单独进行：

```
GPIOC->BSRR = 0x5;           //低16位置位
```

```
GPIOC->BSRR = 0xC0000000;    //高16位复位
```

五. 编程题

1. GPIO口初始化

```
(1) void GPIO_init()
{
    RCC->APB2ENR|=1<<3;    //使能 PORTB 时钟
    GPIOB->CRL&=0X00000000;//清掉原来的设置，同时不影响其它位设置。
    GPIOB->CRL|=0X33333333;//PB 低八位推挽输出
    RCC->APB2ENR|=1<<4;    //使能 PORTC 时钟
    GPIOC->CRL&=0XFFFFFFF0;//清掉原来的设置，同时不影响其它位设置。
    GPIOC->CRL|=0X33;//PB 低八位推挽输出
}
```

2. Lite OS 任务创建

```
/******
 * @ 函数名 : Creat_Test1_Task
 * @ 功能说明: 创建Test1_Task任务
 * @ 参数 :
 * @ 返回值 : 无
 *****/
static UINT32 Creat_Test1_Task()
{
    //定义一个创建任务的返回类型，初始化为创建成功的返回值
    UINT32 uwRet = LOS_OK;
    //定义一个用于创建任务的参数结构体
    TSK_INIT_PARAM_S task_init_param;
    task_init_param.usTaskPrio = 5; /* 任务优先级，数值越
    小，优先级越高 */
    task_init_param.pcName = "Test1_Task"; /* 任务名 */
    task_init_param.pfnTaskEntry =
    (TSK_ENTRY_FUNC)Test1_Task;<—一致
    task_init_param.uwStackSize = 1024; /* 栈大小 */
    uwRet = LOS_TaskCreate(&Test1_Task_Handle,
    &task_init_param);
    return uwRet;
}
```